

Authentication & Authorization Protocols

for IAM & Identity Governance

Staff / Principal Security Architect & Engineer Interview Preparation
Deep technical reference with real-world implementation scenarios

[SAML 2.0](#) | [OAuth 2.0](#) | [OpenID Connect](#) | [JWT](#) | [WebAuthn / FIDO2](#) | [SCIM](#) | [Kerberos](#)

Covers: AWS IAM, GCP Cloud Identity, Azure AD/Entra ID, Okta, Kubernetes RBAC, Zero Trust
2025 Edition

1. The Protocol Landscape: Mental Model

These protocols are not competing alternatives — they operate at **different layers of the identity stack**. Confusing them is the most common mistake in interviews. This section establishes the mental model before diving into each protocol.

Core distinction: Authentication answers "Who are you?" (SAML, OIDC, WebAuthn). Authorization answers "What can you do?" (OAuth 2.0). JWT is a token format used by both. SCIM provisions and deprovisions user accounts. Kerberos authenticates within a trusted network domain.

Protocol	Layer	Core Purpose	Data Format	Year
SAML 2.0	Authentication + SSO	Enterprise web single sign-on across SAML-enabled service providers	XML	2005
OAuth 2.0	Authorization (delegation)	Delegated access: allow app X to access resource Y on your behalf	JSON	2012
OpenID Connect	Authentication + Authorization	Identity layer on top of OAuth 2.0 — who you are + what you can access	JSON + JWT	2014
JWT	Token format	Compact, self-contained, signed data envelope — not a protocol itself	Base64 JSON	2015
WebAuthn / FIDO2	Credential mechanism	Passwordless, phishing-resistant authentication using device hardware	CBOR + JSON	2019
SCIM	Provisioning	Automated user lifecycle management (create, update, disable, delete)	JSON / REST	2015
Kerberos	Authentication (network)	Mutual authentication within a trusted realm using ticket-granting tickets	ASN.1/DER	1988

How they work together — real-world example: In a typical enterprise login to Salesforce via Okta: the user authenticates with WebAuthn (FIDO2 key tap) at Okta. Okta issues a SAML assertion (XML document) containing the user's identity. Salesforce validates the assertion and creates a session. Meanwhile, SCIM has already provisioned the user's Salesforce account from Okta's directory. If the user accesses Salesforce's API, they use OAuth 2.0 tokens (formatted as JWTs) for authorization. All five protocols work together, not instead of each other.

2. SAML 2.0 — Security Assertion Markup Language

SAML 2.0 is an XML-based open standard for exchanging authentication and authorization data between an **Identity Provider (IdP)** and a **Service Provider (SP)**. Published in 2005 by OASIS, it remains the dominant enterprise SSO protocol with support from virtually every enterprise SaaS application.

Layman analogy: Analogy: SAML is a corporate visitor badge system. You visit Security HQ (the IdP — Okta, Azure AD) once, prove your identity, and receive a signed badge (SAML Assertion). You walk into Building A (Salesforce), Building B (Workday), Building C (ServiceNow) — each guard looks at the badge, verifies the HQ stamp, and lets you in without re-checking your passport. One verification, multiple buildings.

2.1 Core Components

Component	Role	Real-World Example
Identity Provider (IdP)	Authenticates users and issues SAML assertions	Okta, Azure AD (Entra ID), OneLogin, PingIdentity, Google Workspace
Service Provider (SP)	Relies on IdP for authentication; consumes SAML assertions	Salesforce, Workday, ServiceNow, Slack, Jira, AWS Console
SAML Assertion	Signed XML document containing subject identity, attributes, conditions, and authentication context	XML with NameID, AudienceRestriction, AuthnStatement, AttributeStatement
Assertion Consumer Service (ACS)	The SP's endpoint that receives and processes the SAML response	https://acme.my.salesforce.com/saml/acs
Metadata	XML file describing the IdP or SP's endpoints, certificates, and supported bindings	Published at /.well-known/saml-metadata or exchanged manually

2.2 SP-Initiated Flow (Most Common)

This is the flow triggered when a user navigates directly to the application (e.g., typing `salesforce.com` into the browser). Over 90% of real-world SAML integrations use this flow.

Step-by-step walkthrough

Step 1: User visits the SP. Jane opens `acme.my.salesforce.com`. She has no session. Salesforce checks its SAML configuration and identifies Okta as the IdP.

Step 2: SP generates AuthnRequest. Salesforce creates a signed XML `AuthnRequest` specifying: the SP's entity ID (Issuer), the ACS URL (where the response should be sent), a unique request ID, and the current timestamp. This is base64-encoded.

Step 3: Browser redirect to IdP. Salesforce returns an HTTP 302 redirect to Okta's SSO endpoint with the `AuthnRequest` as a query parameter:

<https://acme.okta.com/app/saml/sso?SAMLRequest=base64data>

Step 4: User authenticates at IdP. If Jane has no existing Okta session, she sees the Okta login page. She enters credentials and completes MFA (e.g., FIDO2 key tap). If she already has an Okta session cookie (from logging into Slack earlier), this step is invisible — this is the SSO experience.

Step 5: IdP generates SAML Response. Okta constructs a SAML Response containing a signed Assertion with: the NameID (`jane.doe@acme.com`), authentication context (password + MFA), validity window (NotBefore/NotOnOrAfter, typically 5 minutes), AudienceRestriction (only valid for Salesforce's entity ID), and optional attributes (department, role, groups).

Step 6: Browser POST to SP's ACS. Okta returns an HTML form that auto-submits (via JavaScript) to Salesforce's ACS URL. The SAML Response is in a hidden form field. The browser is the courier — it carries the signed assertion but cannot forge it.

Step 7: SP validates and creates session. Salesforce validates: (a) the XML signature using Okta's public X.509 certificate (pre-configured), (b) the assertion is within the NotBefore/NotOnOrAfter window, (c) the AudienceRestriction matches its own entity ID, (d) the InResponseTo matches the original `AuthnRequest` ID (prevents replay). If all checks pass, Salesforce creates a session cookie. Jane is logged in.

Security property: The browser never sends credentials to the SP. The SP never sees the user's password or MFA interaction. The IdP never tells the SP what MFA method was used (unless configured in AuthnContext). This separation of concerns is the fundamental security property of SAML.

2.3 The SAML Assertion — Anatomy

```
<samlp:Response ID="_resp456" InResponseTo="_req123" Version="2.0">
  <saml:Assertion ID="_assert789" IssueInstant="2024-11-15T14:22:00Z">
    <saml:Issuer>https://acme.okta.com</saml:Issuer>
    <ds:Signature>
      <ds:SignedInfo>
        <ds:Reference URI="#_assert789">
          <ds:DigestValue>abc123...</ds:DigestValue>
        </ds:Reference>
      </ds:SignedInfo>
      <ds:SignatureValue>RSA-SHA256-signed-hash...</ds:SignatureValue>
    </ds:Signature>
    <saml:Subject>
      <saml:NameID Format="emailAddress">jane.doe@acme.com</saml:NameID>
      <saml:SubjectConfirmation Method="bearer">
        <saml:SubjectConfirmationData
          Recipient="https://acme.my.salesforce.com/saml/acs"
          NotOnOrAfter="2024-11-15T14:27:00Z"
          InResponseTo="_req123"/>
      </saml:SubjectConfirmation>
    </saml:Subject>
    <saml:Conditions NotBefore="2024-11-15T14:22:00Z"
      NotOnOrAfter="2024-11-15T14:27:00Z">
      <saml:AudienceRestriction>
        <saml:Audience>https://acme.my.salesforce.com</saml:Audience>
      </saml:AudienceRestriction>
    </saml:Conditions>
    <saml:AuthnStatement AuthnInstant="2024-11-15T14:22:00Z">
      <saml:AuthnContext>
        <saml:AuthnContextClassRef>
          urn:oasis:names:tc:SAML:2.0:ac:classes:PasswordProtectedTransport
        </saml:AuthnContextClassRef>
      </saml:AuthnContext>
    </saml:AuthnStatement>
    <saml:AttributeStatement>
      <saml:Attribute Name="groups">
        <saml:AttributeValue>Sales</saml:AttributeValue>
        <saml:AttributeValue>CRM-Admins</saml:AttributeValue>
      </saml:Attribute>
    </saml:AttributeStatement>
  </saml:Assertion>
</samlp:Response>
```

2.4 SAML Security Vulnerabilities

Vulnerability	Description	Real-World Impact	Mitigation
XML Signature	Attacker moves the signed assertion	CVE-2017-11427 (OneLogin Ruby SAML library): attacker	Use canonical XML parsing. Validate that the Signature Reference URI

Vulnerability	Description	Real-World Impact	Mitigation
Wrapping (XSW)	within the XML tree and injects a forged assertion that the signature doesn't cover. The SP validates the signature (which is valid) but processes the forged assertion.	could forge SAML assertions and impersonate any user. Affected thousands of SPs.	matches the Assertion ID. Use mature, audited SAML libraries (never roll your own XML signature validation).
Assertion Replay	Attacker intercepts a valid SAML assertion and replays it at the ACS after the original session	Session hijacking if SP doesn't track consumed assertion IDs	SP must: (1) enforce NotOnOrAfter expiry (5-min window), (2) maintain a cache of consumed assertion IDs and reject duplicates, (3) validate InResponseTo matches the original AuthnRequest ID
Open Redirect in ACS	Attacker manipulates the RelayState parameter to redirect user to a phishing site after successful SAML authentication	Credential harvesting after legitimate login flow	Validate RelayState against an allowlist of permitted redirect URLs. Never use user-supplied URLs in post-authentication redirects.
IdP Certificate Rollover Gap	During certificate rotation at the IdP, SPs still configured with the old certificate reject valid assertions, causing outage. Or worse: the old certificate is compromised but still trusted.	SSO outage across all SPs, or certificate compromise leading to assertion forgery	Support multiple simultaneous signing certificates at the SP. Monitor certificate expiry 90 days ahead. Use SAML metadata auto-refresh (IdP publishes new cert in metadata before rollover).

2.5 Real-World Implementation Scenarios

Scenario A: 200-Account AWS Organization SSO via SAML

An enterprise with 200 AWS accounts uses Okta as IdP. AWS IAM Identity Center is configured as the SAML SP. Okta provisions Permission Sets (Developer, ReadOnly, Admin) via SCIM to IAM Identity Center, which maps them to IAM roles in each account. When an engineer clicks "AWS Console" in Okta, a SAML assertion is POSTed to IAM Identity Center's ACS URL. The assertion contains group memberships (Engineering, SRE, Security). IAM Identity Center maps group membership to Permission Sets, and the engineer lands in the AWS Console with the least-privileged role matching their group. No IAM users or static access keys exist in any account.

Scenario B: B2B Federation Between Two Companies

Acme Corp (customer) needs contractors from Vendor Corp to access Acme's internal Jira. Instead of creating Jira accounts for each contractor, Acme configures Vendor Corp's ADFS as a trusted IdP in Acme's Okta via SAML federation. Vendor Corp's engineers authenticate with their own AD credentials, and the SAML assertion includes an attribute "vendorRole=contractor". Acme's Jira uses attribute-based access control to grant read-only project access. When a contractor leaves Vendor Corp, their AD account is disabled, immediately invalidating all SAML sessions — no action required from Acme's IT.

3. OAuth 2.0 — Authorization Framework

Critical interview point: OAuth 2.0 is NOT an authentication protocol. It is an authorization framework. It tells a service "this token grants access to these resources" but it does NOT tell the service "who the user is." A stolen access token works regardless of who presents it. This is the single most important distinction in the protocol landscape.

OAuth 2.0 (RFC 6749, 2012) solves a specific problem: **how do you let a third-party application access your resources without sharing your password?** Before OAuth, if Canva wanted to access your Google Drive files, you had to give Canva your Google password. OAuth introduces the concept of scoped, revocable access tokens issued by the resource owner (you) via the authorization server (Google).

Layman analogy: Analogy: You check into a hotel (Google, the Authorization Server). The front desk gives you a key card (access token) that opens your room, the gym, and the pool — but NOT the staff office or other guests' rooms. You give this card to the valet (Canva, the client app) so they can access your gym locker. The valet never sees your passport or room code — they only get the key card with limited access. You can revoke the card anytime, and it expires at checkout.

3.1 OAuth 2.0 Roles

Role	Description	Example
Resource Owner	The user who owns the data and grants access	You (your Google account)
Client	The application requesting access to the resource	Canva, Slack, a mobile app, a CI/CD pipeline
Authorization Server (AS)	Issues tokens after authenticating the resource owner and obtaining consent	Google's OAuth server, Okta, Auth0, AWS Cognito
Resource Server	Hosts the protected resources and validates access tokens	Google Drive API, GitHub API, your company's microservices

3.2 Grant Types — When to Use Each

Grant Type	Use Case	Security Level	Flow
Authorization Code + PKCE	Web apps, mobile apps, SPAs — the default for almost all modern applications	Highest for interactive users	User redirected to AS → logs in → AS returns auth code → app exchanges code + code_verifier for tokens via back-channel
Client Credentials	Machine-to-machine: CI/CD pipelines, microservice-to-microservice, cron jobs — no user involved	High (secret-based)	App sends client_id + client_secret directly to token endpoint → receives access token. No user interaction.
Device Code	Input-constrained devices: smart TVs, CLI tools, IoT devices that can't open a browser	Medium	Device displays a code (e.g., WDJB-MJHT) → user enters it at a URL on their phone → device polls the AS until approved
Refresh Token	Extending access without re-prompting the user — used alongside auth code grant	High (must be stored securely)	App sends refresh_token to token endpoint → receives new access_token (and optionally a rotated refresh_token)

Deprecation notice: The Implicit Grant (returning tokens directly in the URL fragment) is deprecated in OAuth 2.1 (draft). Never use it for new applications. The Authorization Code grant with PKCE (Proof Key for Code Exchange, RFC 7636) is the replacement for all client types, including SPAs and mobile apps. PKCE prevents authorization code interception attacks by requiring the client to prove it initiated the flow using a cryptographic code_verifier/code_challenge pair.

3.3 Authorization Code + PKCE — Step by Step

```
# 1. Client generates PKCE values (one-time, per-flow)
code_verifier = random_string(43_to_128_chars) # stored client-side
code_challenge = BASE64URL(SHA256(code_verifier)) # sent to AS

# 2. Redirect user to authorization endpoint
GET https://accounts.google.com/o/oauth2/v2/auth?
  response_type=code
  &client_id=canva-app-id
  &redirect_uri=https://canva.com/callback
  &scope=https://www.googleapis.com/auth/drive.readonly
  &state=random_csrf_token # prevents CSRF on the callback
  &code_challenge=E9Melhoa2... # SHA256 hash of code_verifier
  &code_challenge_method=S256

# 3. User authenticates and consents. Google redirects back:
302 https://canva.com/callback?code=AUTH_CODE_123&state=random_csrf_token

# 4. Client exchanges code for tokens (back-channel, server-to-server)
POST https://oauth2.googleapis.com/token
  grant_type=authorization_code
  &code=AUTH_CODE_123
  &redirect_uri=https://canva.com/callback
  &client_id=canva-app-id
  &client_secret=CLIENT_SECRET # web apps only; mobile apps omit
  &code_verifier=original_random_string # proves this client started the flow

# 5. Response contains tokens
{
  "access_token": "ya29.a0ARrdaM...", // 1-hour expiry typically
  "token_type": "Bearer",
  "expires_in": 3600,
  "refresh_token": "1//0eFQmR...", // for long-lived access
  "scope": "https://www.googleapis.com/auth/drive.readonly"
}
```

3.4 Real-World Implementation Scenarios

Scenario A: GitHub Actions OIDC → AWS STS (No Stored Credentials)

A GitHub Actions workflow needs to deploy to AWS. Instead of storing AWS access keys as GitHub secrets (a credential exposure risk), the workflow uses GitHub's OIDC provider. GitHub Actions issues an OIDC token (a JWT) asserting the repository, branch, and workflow identity. The workflow calls AWS STS AssumeRoleWithWebIdentity, passing the GitHub OIDC token. AWS validates the token against GitHub's OIDC discovery document (/.well-known/openid-configuration), checks the audience and subject claims against the IAM role's trust policy conditions, and returns temporary STS credentials (15-minute to 1-hour

expiry). No long-lived credentials are ever stored. This is the OAuth 2.0 client_credentials pattern adapted for CI/CD via OIDC federation.

Scenario B: Microservice-to-Microservice Authorization

An order service needs to call the payment service. Both run in Kubernetes. The order service uses a ServiceAccount token (Kubernetes issues a JWT via the TokenRequest API) to authenticate to the company's internal OAuth 2.0 server (Keycloak or Auth0). The OAuth server validates the K8s JWT, looks up the order-service's registered scopes, and issues an access token with scope=payments:charge. The payment service validates this access token (by calling the OAuth server's introspection endpoint or by validating the JWT signature locally using the JWKS endpoint) and verifies the scope includes payments:charge before processing the request. No service account passwords or static API keys exist.

4. OpenID Connect (OIDC)

OIDC (2014) is an **identity layer built on top of OAuth 2.0**. It adds a standardized way to obtain user identity information — something OAuth 2.0 deliberately does not provide. While OAuth 2.0 gives you a key card (access token), OIDC staples your photo ID (ID token) to it.

Layman analogy: Analogy: OAuth 2.0 = hotel key card (opens gym, pool). OIDC = hotel key card WITH your photo ID laminated on top. The card still opens the same doors, but now the valet can also verify you're the guest — not just someone who found a card on the floor.

4.1 What OIDC Adds to OAuth 2.0

Feature	OAuth 2.0	OIDC Addition
Answers the question	"What can you access?" (authorization)	"Who are you?" + "What can you access?" (authentication + authorization)
Token types	Access token (opaque or JWT)	Access token + ID Token (always a JWT) + optional Refresh Token
User identity	Not standardized — each provider has a different /userinfo API shape	Standardized ID Token claims: sub (unique ID), email, name, picture, iss, aud, exp
Discovery	No standard	/.well-known/openid-configuration — JSON document listing all endpoints, supported scopes, signing algorithms
Scopes	Provider-specific	Standard scopes: openid (required), profile, email, address, phone
Session management	Not defined	Front-channel and back-channel logout, session management via iframes

4.2 The ID Token — A Decoded JWT

The ID Token is always a JWT (JSON Web Token). When you decode it (base64), you see a standard set of claims:

```
// Header (algorithm + key ID)
{
  "alg": "RS256",
  "kid": "fle2d3c4",    // key ID used for signing (maps to JWKS endpoint)
  "typ": "JWT"
}

// Payload (identity claims)
{
  "iss": "https://accounts.google.com",           // who issued this token
  "sub": "110248495921238986420",                // unique, stable user ID
  "aud": "your-app-client-id.apps.google.com",    // intended audience (YOUR app)
  "exp": 1700000000,                             // expiry (Unix timestamp)
  "iat": 1699996400,                             // issued at
  "nonce": "abc123random",                       // replay protection
  "email": "jane@acme.com",
  "email_verified": true,
  "name": "Jane Doe",
  "picture": "https://lh3.googleusercontent.com/...",
  "hd": "acme.com"                               // hosted domain (Google Workspace)
```

```
}  
  
// Signature  
RSASHA256(base64url(header) + '.' + base64url(payload), google_private_key)
```

Validation checklist: Validation checklist that every SP must perform: (1) Verify the JWT signature against the issuer's JWKS endpoint keys (not a static key — keys rotate). (2) Check iss matches expected issuer. (3) Check aud matches your app's client_id. (4) Check exp > current time. (5) Check nonce matches what you sent in the auth request. (6) Check iat is within a reasonable clock skew window. Skipping ANY of these allows token forgery or replay.

4.3 Real-World OIDC Implementation Scenarios

Scenario A: GKE Workload Identity (Pod → BigQuery with zero credentials on disk)

A GKE pod needs to query BigQuery. Instead of mounting a service account JSON key file (which is long-lived, exportable, and a top GCP attack vector), the cluster uses Workload Identity. The GKE metadata server (DaemonSet) intercepts IMDS requests from the pod and issues a short-lived OIDC token scoped to the pod's Kubernetes ServiceAccount. The pod's GCP client library sends this OIDC token to sts.googleapis.com (Security Token Service), which validates it against the GKE cluster's OIDC issuer (discoverable via /.well-known/openid-configuration). STS returns a Google access token valid for 1 hour, scoped to the IAM bindings on the mapped GCP ServiceAccount. Cloud Audit Logs record the GCP SA email plus the Kubernetes namespace and ServiceAccount in the principalEmail field — full attribution, no key file, automatic rotation.

Scenario B: AWS IRSA (IAM Roles for Service Accounts in EKS)

Same pattern, different cloud. An EKS pod needs S3 access. The EKS cluster has an OIDC provider registered with AWS IAM. The pod's Kubernetes ServiceAccount is annotated with the IAM role ARN. When the pod's SDK requests credentials, it calls the EKS Pod Identity webhook, which injects AWS credentials into the pod environment. Under the hood, the webhook calls STS AssumeRoleWithWebIdentity using the pod's projected OIDC token. The IAM role's trust policy validates: (a) the OIDC provider URL matches the EKS cluster, (b) the sub claim matches the specific K8s namespace/ServiceAccount. This is OIDC federation — Kubernetes is the IdP, AWS is the SP.

5. JWT — JSON Web Tokens

Critical interview point: JWT is NOT a protocol. It is a token format. Saying "we use JWT for authentication" is like saying "we use envelopes for mail delivery." The envelope (JWT) carries the message; the protocol (OIDC, OAuth 2.0) defines who sends it, where, and how it's validated.

JWT (RFC 7519) is a compact, URL-safe means of representing claims between two parties. It consists of three base64url-encoded parts separated by dots: Header.Payload.Signature.

5.1 JWT Security Vulnerabilities

Vulnerability	How It Works	Real-World CVE	Prevention
alg:none attack	Attacker sets the algorithm header to "none", making the signature empty. Libraries that trust the header's algorithm claim skip verification entirely.	CVE-2015-9235 (jwt-simple for Node.js): allowed alg:none, enabling any user to forge tokens as any other user	NEVER trust the alg claim from the token itself. Configure a server-side allowlist of accepted algorithms (e.g., RS256 only). Reject tokens with alg:none.
Key confusion (RSA/HMAC)	Server is configured to verify with RSA public key. Attacker switches alg from RS256 to HS256 and signs with the RSA public key (which is public). Server uses the same key material for HMAC verification — and it passes.	CVE-2016-5431 (Auth0 jsonwebtoken): attacker could forge tokens by signing with the known public key	Enforce algorithm per-key, not per-token. If you have an RSA key pair, only accept RS256/RS384/RS512 for that kid. Never use the same key for both HMAC and RSA.
Token lifetime abuse	JWT with a 24-hour expiry is stolen. Attacker has 24 hours of valid access because JWTs are stateless — there is no server-side revocation.	Common in API gateway architectures where JWTs are validated locally	Use short-lived access tokens (5–15 minutes). Use refresh tokens for session extension. For critical actions, require re-authentication (step-up auth).
Payload information leakage	Developers put sensitive data (internal user IDs, permissions, API keys) in the JWT payload, forgetting it's only base64-encoded — not encrypted.	Recurring pattern in security audits and bug bounty reports	Never put secrets in JWTs. If you need encrypted payloads, use JWE (JSON Web Encryption). Treat JWT payloads as public data.

6. WebAuthn / FIDO2 — Passwordless Authentication

WebAuthn (W3C standard, 2019) is the web API that enables **phishing-resistant, passwordless authentication** using hardware authenticators (YubiKeys, platform authenticators like Touch ID and Windows Hello, and passkeys synced via iCloud Keychain or Google Password Manager). It is part of the FIDO2 framework, which also includes CTAP2 (Client-to-Authenticator Protocol).

Layman analogy: Analogy: Your front door has a special smart lock. When you first moved in, you registered your unique fingerprint (enrolled a credential). To enter, you touch the reader — the lock sends a mathematical challenge, your fingerprint sensor signs it with a private key that never leaves the device. A lockpick who builds a perfect replica of your door somewhere else (a phishing site) fails because your fingerprint sensor only responds to challenges from YOUR lock's specific domain. No password to steal, no code to intercept, no credential to phish.

6.1 Why WebAuthn Is Phishing-Resistant

This is the most asked follow-up question in interviews. The answer is origin binding.

Origin binding: When the authenticator (YubiKey, Touch ID) signs the challenge, it includes the website's origin (domain) in the signed data. The credential is cryptographically bound to "login.acme.com". If an attacker creates a fake site at "login-acme.com" (a phishing domain), the browser sends "login-acme.com" as the origin. The authenticator either (a) doesn't have a credential for that domain, or (b) signs with the wrong domain in the signed data, causing server-side validation to fail. The user cannot accidentally authenticate to the wrong site.

Compare this with TOTP (Google Authenticator), which is NOT phishing-resistant: the attacker proxies the TOTP code in real-time from the phishing page to the real site. With WebAuthn, there is nothing to proxy — the cryptographic challenge is bound to the domain, not to a shared secret.

6.2 Real-World Implementation Scenarios

Scenario A: Eliminating Phishable MFA for AWS Console Access

An enterprise mandates that all AWS Console access requires WebAuthn (FIDO2) as the second factor, replacing TOTP. In AWS IAM Identity Center (or Okta), the admin configures MFA policy to require FIDO2 security keys. Engineers register their YubiKeys. When accessing the AWS Console, the engineer enters their password at Okta, and then the browser triggers `navigator.credentials.get()`. The YubiKey blinks, the engineer taps it, and the signed challenge is sent to Okta. Because the challenge includes `okta.com` as the origin, a phishing page at `Okta.com` would fail. This eliminated two MFA-bypass phishing incidents the enterprise experienced quarterly with TOTP-based MFA.

Scenario B: Passkeys for Customer Authentication

A fintech app implements passkeys (synced WebAuthn credentials via iCloud Keychain / Google Password Manager) for customer login. During registration, the app calls `navigator.credentials.create()` with the user's email as the user handle. The platform authenticator (Face ID, Windows Hello) generates a key pair. The public key is stored on the server. For login, the app calls `navigator.credentials.get()` — the user touches Face ID, the private key signs the challenge, and the server validates. No password, no SMS OTP, no phishing risk. The credential syncs across the user's Apple/Google devices via end-to-end encrypted sync.

7. SCIM — Automated User Provisioning

SCIM (System for Cross-domain Identity Management, RFC 7643/7644) is the **REST API standard for automating user account lifecycle management** across systems. While SAML and OIDC handle how a user logs in, SCIM handles how the user's account is created, updated, and disabled in each application.

Layman analogy: Analogy: SAML is the badge that gets you through the door. SCIM is the HR process that creates your badge record in every building's security system on your first day, updates your access when you transfer departments, and deactivates the badge across all buildings on your last day.

7.1 SCIM in Practice

When an admin creates a user in Okta and assigns them Salesforce, Okta's SCIM client calls Salesforce's SCIM endpoint:

```
POST https://acme.my.salesforce.com/services/scim/v2/Users
Authorization: Bearer <OAuth 2.0 token>
Content-Type: application/scim+json

{
  "schemas": ["urn:ietf:params:scim:schemas:core:2.0:User"],
  "userName": "jane.doe@acme.com",
  "name": { "givenName": "Jane", "familyName": "Doe" },
  "emails": [{ "value": "jane.doe@acme.com", "primary": true }],
  "active": true,
  "groups": [{ "value": "CRM-Users" }]
}
```

When Jane is terminated, Okta sends a PATCH request setting active: false. Salesforce immediately deactivates the account. No manual ticket, no 24-hour lag, no orphaned account. This is why SCIM is critical for IGA (Identity Governance and Administration) — it is the enforcement mechanism for the Joiner-Mover-Leaver lifecycle.

7.2 Interview Scenario: Detecting Provisioning Drift

Your SCIM integration with ServiceNow is failing silently — the SCIM endpoint returns 200 OK but doesn't actually deactivate users. How do you detect this? Answer: Implement provisioning reconciliation. Nightly cron job calls ServiceNow's SCIM GET /Users endpoint and compares the active user list against Okta's user store. Any user who is inactive in Okta but active in ServiceNow triggers an alert and automatic remediation (SCIM PATCH to deactivate). Log all SCIM operations to your SIEM (Splunk, Chronicle) with correlation IDs linking to the originating HR event.

8. Kerberos — Network Authentication Protocol

Kerberos (MIT, 1988, v5 is current) provides **mutual authentication within a trusted network realm** using symmetric-key cryptography and a trusted third party called the Key Distribution Center (KDC). It is the default authentication protocol for Active Directory, and therefore underpins the majority of on-premises enterprise authentication worldwide.

Layman analogy: Analogy: You're at an amusement park. Instead of buying a ticket at every ride (entering your password at every service), you go to the main ticket booth (KDC) once and buy a day pass (Ticket-Granting Ticket / TGT). At each ride (service), you show your day pass to the ride's ticket window (TGS), which gives you a ride-specific ticket (service ticket). The ride operator validates this ticket without calling the main booth again. The day pass and ride tickets are time-limited and non-transferable.

8.1 Kerberos Flow (Simplified)

Step	Action	What Happens Technically
1. AS-REQ	Client → KDC Authentication Service	Client sends username + encrypted timestamp (pre-authentication). KDC verifies against its database.
2. AS-REP	KDC → Client	KDC returns: (a) TGT encrypted with the KDC's secret (client can't read it), (b) session key encrypted with client's password hash (client decrypts with their password).
3. TGS-REQ	Client → KDC Ticket-Granting Service	Client sends TGT + authenticator (encrypted with session key) + requested service name (SPN).
4. TGS-REP	KDC → Client	KDC returns a service ticket encrypted with the target service's secret key + a new session key for client-service communication.
5. AP-REQ	Client → Service	Client sends the service ticket + authenticator to the target service. Service decrypts with its own secret key, validates, and grants access.

8.2 Kerberos Attack Patterns (Staff+ Must Know)

Attack	Mechanism	Detection	Mitigation
Kerberoasting	Request service tickets for accounts with SPNs, then crack the ticket offline (service ticket is encrypted with the service account's password hash). If the service account has a weak password, cracking takes minutes.	Anomalous TGS requests for multiple SPNs from a single user. 4769 events with encryption type 0x17 (RC4).	Use Group Managed Service Accounts (gMSAs) with 120-character auto-rotating passwords. Set minimum 25-char passwords for manual service accounts. Monitor for 4769 events with RC4 encryption.
Golden Ticket	Attacker compromises the KRBTGT account's password hash (requires domain admin). Can forge TGTs for any user with any privileges for up to 10 years (default TGT lifetime).	TGTs with unusual lifetimes. Authentication events without corresponding AS-REQ events.	Rotate KRBTGT password twice (the second rotation invalidates the old hash). Implement Privileged Access Workstations (PAWs). Monitor for TGTs created outside of normal KDC operations.
Silver Ticket	Attacker compromises a service account's password hash. Can forge	Service access without corresponding TGS-REQ events in the KDC	Enable PAC validation on services. Use gMSAs. Monitor for service access events not preceded by TGS events.

Attack	Mechanism	Detection	Mitigation
	service tickets for that specific service without contacting the KDC.	logs. PAC validation failures.	
Pass-the-Ticket	Attacker extracts TGT or service tickets from memory (using Mimikatz/Rubeus) and injects them into another session on a different machine.	Same ticket used from different IPs. Logon events from unexpected hosts.	Enable Credential Guard on Windows 10/11. Use Protected Users security group. Implement network segmentation limiting lateral movement.

9. How These Protocols Interconnect

This is the section most interview candidates miss. Each protocol in isolation is well-documented, but the **interaction patterns between them** are where real-world architecture lives. Interviewers at Staff/Principal level probe whether you understand how these protocols chain together, where handoffs occur, and what breaks when one link fails.

9.1 The Protocol Stack — Layered Architecture

These protocols operate at distinct layers, and each layer depends on the one below it:

Layer	Protocol(s)	What It Does	Depends On
L5: Governance	IGA Platforms (SailPoint, Saviynt)	Access certification, SoD enforcement, audit reporting	Layer 4 data (who has what)
L4: Provisioning	SCIM 2.0	Creates/updates/disables user accounts and group memberships	Layer 3 identity store (IdP)
L3: Authentication	SAML 2.0, OIDC, Kerberos	Proves user identity. Issues assertions/tokens.	Layer 2 credential verification
L2: Credential	WebAuthn/FIDO2, Passwords, OTP, Certificates	The mechanism by which the user proves identity	Layer 1 user record
L1: Directory	LDAP, Active Directory, Cloud Identity	Stores user attributes, groups, credential material	HR system (source of truth)
Cross-cutting	OAuth 2.0 (authz) + JWT (format)	Grants scoped access tokens; carries claims as signed envelopes	Layer 3 authentication context

9.2 Protocol Chaining — How They Build on Each Other

Chain 1: OIDC is OAuth 2.0 + Identity

OIDC does not replace OAuth 2.0 — it extends it. An OIDC flow IS an OAuth 2.0 flow with the `openid` scope added. The authorization server returns both an access token (OAuth 2.0's output) and an ID token (OIDC's addition, a JWT with identity claims). The same `/authorize` and `/token` endpoints serve both. Removing the `openid` scope from the request reduces the flow to pure OAuth 2.0 — no ID token is returned.

Chain 2: WebAuthn feeds into SAML or OIDC

WebAuthn is a credential verification mechanism, not a protocol for transmitting identity assertions. It operates at the IdP during authentication. When you tap a YubiKey on Okta's login page, WebAuthn handles the cryptographic challenge-response. Once Okta verifies the WebAuthn signature, it issues either a SAML Assertion (for SAML-configured SPs) or an OIDC ID token (for OIDC SPs). The SP never sees the WebAuthn interaction — it only sees the SAML/OIDC output. WebAuthn is the *input* to authentication; SAML/OIDC is the *output*.

Chain 3: SCIM runs parallel to SAML/OIDC (not sequential)

SCIM is an out-of-band provisioning protocol that operates asynchronously. When an admin assigns a user to Salesforce in Okta, SCIM creates the account via REST API hours or days before the user ever logs in. When the user logs in via SAML, Salesforce already has their account. If SCIM fails (e.g., SP

endpoint down), the SAML assertion is valid but the user has no account — a common operational failure. Mitigation: implement JIT (Just-In-Time) provisioning as fallback.

Chain 4: OAuth 2.0 tokens are often JWTs

OAuth 2.0 does not mandate a token format — access tokens can be opaque strings (validated by introspection endpoint) or self-contained JWTs (validated by signature using JWKS). AWS Cognito, Auth0, and Okta issue JWT-formatted access tokens by default. The OIDC ID token is always a JWT by specification. When a service receives a JWT access token, it validates locally using the JWKS endpoint — no network call to introspection. This trades consistency for latency.

Chain 5: Kerberos bridges to SAML/OIDC at the perimeter

In hybrid environments, on-premises users authenticate with Kerberos (via AD) but need cloud SaaS apps that speak SAML/OIDC. ADFS or Azure AD Connect bridges this: the user's Kerberos TGT silently authenticates to ADFS, which issues a SAML assertion for the cloud SP. Kerberos handles internal network auth; SAML handles cross-domain federation. The user experiences seamless SSO from domain-joined laptop to Salesforce.

9.3 End-to-End Lifecycle — All Protocols in One Employee Journey

Follow one employee from Day 1 to termination — every protocol in action:

Phase	What Happens	Protocols Active
Day 0: HR creates record	Workday creates Jane's profile (dept=Engineering, title=SRE)	None yet — HR is source of truth
Day 0: Accounts provisioned	Okta receives HR feed, creates identity, auto-assigns groups. SCIM pushes account creation to Salesforce, Jira, Slack, AWS IAM Identity Center, GitHub.	SCIM 2.0 (Okta → each SP). SCIM calls themselves authenticated via OAuth 2.0 client_credentials.
Day 1: MFA enrollment	Jane registers YubiKey and Face ID passkey as authenticators	WebAuthn (FIDO2) — navigator.credentials.create() generates key pairs. Public keys stored in Okta.
Day 1: First SSO login to Slack	Slack redirects to Okta. Jane taps YubiKey. Okta issues SAML assertion. Slack validates and creates session.	WebAuthn (credential) → SAML 2.0 (assertion transport) → JWT (Slack session token)
Day 1: AWS Console access	Jane clicks AWS in Okta dashboard. SAML assertion POSTed to IAM Identity Center. Jane lands in Console with Developer Permission Set.	SAML 2.0 (Okta → IAM Identity Center) → STS AssumeRole (temporary credentials)
Day 1: EKS pod calls S3	Jane's microservice pod uses IRSA to get S3 access. Pod's OIDC token exchanged for STS temp creds.	OIDC (K8s token) → OAuth 2.0 (STS token exchange) → JWT (the OIDC token format)
Month 6: Department transfer	HR updates Workday: dept=Security. Okta receives change, updates group membership. SCIM patches downstream apps.	SCIM 2.0 (Okta → all SPs). Old Engineering access revoked, new Security access granted automatically.
Day Last: Termination	HR marks terminated at 5PM. Okta deactivates within 60 seconds. SCIM PATCH active:false to all SPs. SAML back-channel logout kills all sessions. OAuth refresh tokens invalidated. Kerberos TGT revoked via AD account disable.	SCIM (deactivate) + SAML (session kill) + OAuth (refresh token revocation) + Kerberos (TGT invalidation) + WebAuthn (credentials deregistered)

Gap analysis — the termination timing hole: Interview question: "What is the gap in this termination process?" Answer: OAuth 2.0 access tokens that were already issued before termination remain valid until their expiry (typically 1 hour). If Jane's microservice has an access token with 45 minutes remaining, it continues to work. Mitigation: short-lived access tokens (5–15 min), token introspection (real-time validation), or a deny list of revoked jti claims at the API gateway. This is a fundamental limitation of stateless JWT-based access tokens.

9.4 Cross-Protocol Failure Modes

What breaks when one protocol link in the chain fails:

Failure	Impact	Mitigation
SCIM endpoint down at SP	New accounts not provisioned. SAML login succeeds but user has no account → "user not found" error	Implement JIT provisioning as fallback: SAML assertion attributes auto-create the account on first login. JIT + SCIM = defense in depth.
IdP outage (Okta down)	ALL SSO flows fail. No user can authenticate to any application.	Break-glass local admin accounts on critical SPs. IdP HA (Okta 99.99% SLA). AWS: IAM Identity Center emergency access.
JWKS endpoint unreachable	SPs can't fetch signing keys to validate OIDC/JWT tokens. All OIDC/OAuth flows fail.	Cache JWKS keys locally (24h TTL). Most OIDC libraries do this automatically.
Certificate rollover mismatch	IdP signs with new cert. SPs have old cert. Signature validation fails → SSO broken.	Support multiple simultaneous signing certs. Publish new cert 30 days before activation. SAML metadata auto-refresh.
OAuth refresh token rotation failure	Client gets new access token but fails to persist rotated refresh token (crash). Old one revoked. User must re-auth.	Refresh token reuse detection with short grace period (30s). Auth0 and Okta support this.
Kerberos clock skew > 5 min	Kerberos authentication fails for all users on affected machines	NTP sync on all domain-joined machines. Monitor clock drift as health metric.

9.5 Protocol Interaction Diagram — Single Login Flow

The complete protocol chain for enterprise SSO to a cloud SaaS application:

```

USER (with YubiKey)
|
| 1. Navigates to salesforce.com
v
SALESFORCE (Service Provider)
|
| 2. No session. Generates SAML AuthnRequest.
|   Redirects to Okta. [SAML 2.0]
v
OKTA (Identity Provider + Authorization Server)
|
| 3. No Okta session. Shows login page.
| 4. Triggers WebAuthn challenge. [WebAuthn/FIDO2]
|   User taps YubiKey. Signed with origin=okta.com.
| 5. Auth successful. Generates SAML Assertion.
|   [SAML 2.0 Assertion with groups, NameID]
| 6. Browser POSTs assertion to Salesforce ACS.

```

```
v
SALESFORCE (Service Provider)
|
| 7. Validates signature. Account exists (SCIM pre-provisioned).
|   [SCIM 2.0 ran at Day 0]
|   Creates session.
| 8. User calls Salesforce API.
|   [OAuth 2.0 access token issued, JWT format]
v
SALESFORCE API (Resource Server)
    Validates JWT. Checks scopes. Returns data.

PARALLEL (background, out-of-band):
    OKTA --SCIM--> Salesforce, AWS, Jira, Slack (provisioning)
    OKTA --SCIM--> IGA Platform (access data for certification)
    [All SCIM calls authenticated via OAuth 2.0 client_credentials]
```

Interconnection summary: Five protocols active in one login: (1) SAML carries identity between Okta and Salesforce. (2) WebAuthn handles the cryptographic credential verification. (3) SCIM pre-provisioned the account. (4) OAuth 2.0 grants API access post-login. (5) JWT is the format for both the OIDC session token inside Okta and the Salesforce API access token. None works alone — they are a coordinated stack.

10. Protocol Selection Decision Matrix

In staff/principal interviews, you'll be asked: "which protocol would you use for X?" This matrix maps real scenarios to the correct protocol combination.

Scenario	Primary Protocol	Supporting	Rationale
Enterprise SSO to SaaS apps (Salesforce, Workday, Slack)	SAML 2.0 or OIDC	SCIM for provisioning	SAML has broadest legacy SP support. New integrations prefer OIDC. SCIM automates JML lifecycle.
"Sign in with Google" for a consumer web app	OIDC	OAuth 2.0 (bundled)	OIDC provides standardized identity claims. OAuth provides API access tokens.
CI/CD pipeline deploying to AWS (GitHub Actions, GitLab)	OIDC federation	OAuth 2.0 token exchange	OIDC token from CI provider → AWS STS AssumeRoleWithWebIdentity. No stored credentials.
Kubernetes pod accessing cloud APIs (S3, BigQuery)	OIDC (Workload Identity / IRSA)	OAuth 2.0 token exchange	OIDC token exchange for short-lived cloud credentials. No service account keys.
Mobile app accessing REST API with user context	OIDC + OAuth 2.0 (Auth Code + PKCE)	JWT (token format)	OIDC for user identity, OAuth for API scoped access, PKCE for mobile security.
Microservice-to-microservice in a service mesh	OAuth 2.0 (client_credentials)	mTLS + JWT	Client credentials grant for machine identity. mTLS for transport auth. JWT as access token format.
Phishing-resistant MFA for privileged admin accounts	WebAuthn / FIDO2	OIDC (as transport)	Hardware authenticator (YubiKey) with WebAuthn via the IdP's OIDC flow.
On-premises Active Directory domain authentication	Kerberos	NTLM (fallback only)	Kerberos for all domain-joined systems. NTLM only as fallback for legacy systems (disable if possible).
Automated user provisioning from HR system to 50 SaaS apps	SCIM 2.0	SAML/OIDC (for login)	SCIM handles create/update/deactivate. SAML/OIDC handles actual login. Both required.
B2B partner federation (vendor's employees access your portal)	SAML 2.0 federation	SCIM (optional)	SAML cross-domain federation. Each partner is an IdP. Your portal is the SP. SCIM for JIT provisioning.
Zero Trust architecture for a 500-person company	OIDC + WebAuthn + OAuth 2.0	SCIM, mTLS, JWT	WebAuthn for phishing-resistant authn. OIDC for identity federation. OAuth for API authz. SCIM for lifecycle. mTLS for service mesh.

11. Identity Governance & Administration (IGA)

IGA is the discipline of managing the full lifecycle of digital identities and their entitlements. It answers: who has access to what, who granted it, is it still appropriate, and can we prove it to auditors? The protocols above are the technical implementation layer; IGA is the governance and compliance wrapper.

10.1 Joiner-Mover-Leaver Lifecycle

Phase	HR Event	IGA Action	Protocol Used	Cloud Implementation
Joiner	New hire on Day 1	Create accounts in all required systems with role-appropriate access based on department, title, and location attributes from HRIS	SCIM (provisioning), OIDC/SAML (SSO)	Okta/Azure AD → SCIM to Salesforce, Jira, Slack, AWS IAM Identity Center. Day-1 access ready before laptop arrives.
Mover	Transfer to new department	Revoke old access, grant new role-appropriate access, trigger re-certification for sensitive entitlements	SCIM (update attributes + groups), ABAC policy re-evaluation	HR system pushes department change → Okta group membership updates → SCIM patches downstream apps → AWS Permission Sets change via IAM Identity Center group mapping.
Leaver	Termination (voluntary or involuntary)	Immediately disable all accounts, revoke all active sessions, archive data per retention policy	SCIM (deactivate), session revocation APIs, SAML session kill	HR marks inactive → Okta deactivates → SCIM PATCH active:false to all 50 SPs within 60 seconds. Okta session kill terminates all active SAML sessions. AWS STS tokens cannot be revoked (1-hour max expiry).

10.2 Access Certification & Recertification

SOX, HIPAA, PCI DSS, and SOC 2 audits require periodic access reviews. The governance question is: does Jane in Marketing still need admin access to the production database she was granted 18 months ago during an incident? IGA platforms (SailPoint, Saviynt, Okta Identity Governance) automate this by pulling current entitlements from SCIM/LDAP endpoints, generating review campaigns where managers must certify or revoke each entitlement, and enforcing SoD (Separation of Duties) policies that prevent the same user from having both "approve purchase order" and "process payment" entitlements.

Interview scenario: Interview question: "How do you handle the orphaned account problem across 200 AWS accounts?" Answer: Deploy Okta's SCIM integration with AWS IAM Identity Center. All access is provisioned through Permission Sets mapped to Okta groups. An employee who is not in an Okta group has zero AWS access — no IAM users, no console passwords, no access keys exist outside of Identity Center. Run a monthly reconciliation report comparing Identity Center assignments against CloudTrail's `userIdentity.arn` fields to detect any access not flowing through the IdP (e.g., manually created IAM users).

12. Interview Quick-Reference

11.1 Common Trick Questions

Question	Wrong Answer (Common Mistake)	Correct Answer
"How does OAuth 2.0 authenticate users?"	"OAuth authenticates users through the authorization code flow..."	"OAuth 2.0 does NOT authenticate users. It is an authorization framework. For user authentication, you need OIDC, which adds an ID token (containing user identity claims) on top of OAuth 2.0. The access token in OAuth tells you what you can access, not who you are."
"Is JWT secure?"	"Yes, JWTs are signed so they're secure."	"JWTs are tamper-evident, not tamper-proof. The payload is base64-encoded, not encrypted — anyone can read it. Security depends on: (1) using strong algorithms (RS256, not HS256 with weak keys), (2) validating signatures against a trusted JWKS endpoint, (3) checking audience, issuer, and expiry claims, (4) using short lifetimes (5–15 min for access tokens). JWTs are also not revocable — once issued, they're valid until expiry."
"SAML vs OIDC — which is better?"	"OIDC is newer so it's better."	"Neither is universally better — they're suited for different contexts. SAML excels at enterprise web SSO where XML parsing is acceptable and legacy SP support matters. OIDC is preferred for: new integrations, mobile apps, APIs, machine identity federation (IRSA, Workload Identity), and scenarios where JWT-based tokens are needed. Many enterprises run both simultaneously."
"How do you revoke a JWT?"	"Delete it from the client."	"JWTs are stateless and cannot be revoked once issued. Options: (1) Use short-lived tokens (5-min access + long-lived refresh token). (2) Token introspection endpoint for real-time validation (adds latency). (3) Server-side deny list of jti (JWT ID) claims for compromised tokens (defeats the stateless benefit). (4) Rotate the signing key (invalidates ALL tokens). The correct architectural answer is: use short access tokens + refresh tokens with rotation."
"What's the difference between passkeys and WebAuthn?"	"They're the same thing."	"WebAuthn is the W3C API standard (navigator.credentials.create/get). Passkeys are a specific implementation of WebAuthn where the credential is synced across devices (via iCloud Keychain, Google Password Manager, or Windows Hello). Traditional WebAuthn credentials (e.g., YubiKey) are device-bound and not synced. Passkeys prioritize convenience (sync); hardware keys prioritize security (non-exportable)."

11.2 Protocol-to-Cloud Service Mapping

Protocol	AWS Implementation	GCP Implementation	Azure/Entra ID
SAML 2.0	IAM Identity Center (SSO), IAM SAML Provider for federated console access	Cloud Identity SAML app integration, Google Workspace as IdP	Entra ID Enterprise Applications (SAML SSO), B2B Direct Federation
OAuth 2.0	Cognito User Pools (hosted UI), API Gateway authorizers, ALB OIDC/OAuth	Identity-Aware Proxy (IAP), API Gateway, Cloud Endpoints	Entra ID App Registrations, Microsoft Graph API scopes

Protocol	AWS Implementation	GCP Implementation	Azure/Entra ID
OIDC	IRSA (EKS), IAM OIDC Provider for CI/CD federation, Cognito OIDC	Workload Identity (GKE), Workforce Identity Federation	Entra ID OIDC app registration, Managed Identities
JWT	Cognito tokens, API Gateway JWT authorizer, ALB authentication	Identity Platform (Firebase Auth) tokens, Cloud Endpoints JWT validation	Entra ID tokens (all are JWTs), Azure API Management JWT validation
WebAuthn	IAM FIDO2 MFA for root and IAM users, Identity Center FIDO2	Google Workspace security keys enforcement, Chrome device trust	Entra ID passwordless (FIDO2 security keys), Windows Hello for Business
SCIM	IAM Identity Center SCIM provisioning from Okta/Azure AD	Cloud Identity SCIM provisioning, Google Workspace Directory API	Entra ID SCIM provisioning to 1000+ SaaS gallery apps
Kerberos	AWS Managed Microsoft AD (Kerberos for domain-joined EC2)	N/A (Kerberos not native to GCP; use AD on GCE if needed)	Azure AD DS (Kerberos for Azure-hosted VMs), Entra ID Kerberos for hybrid

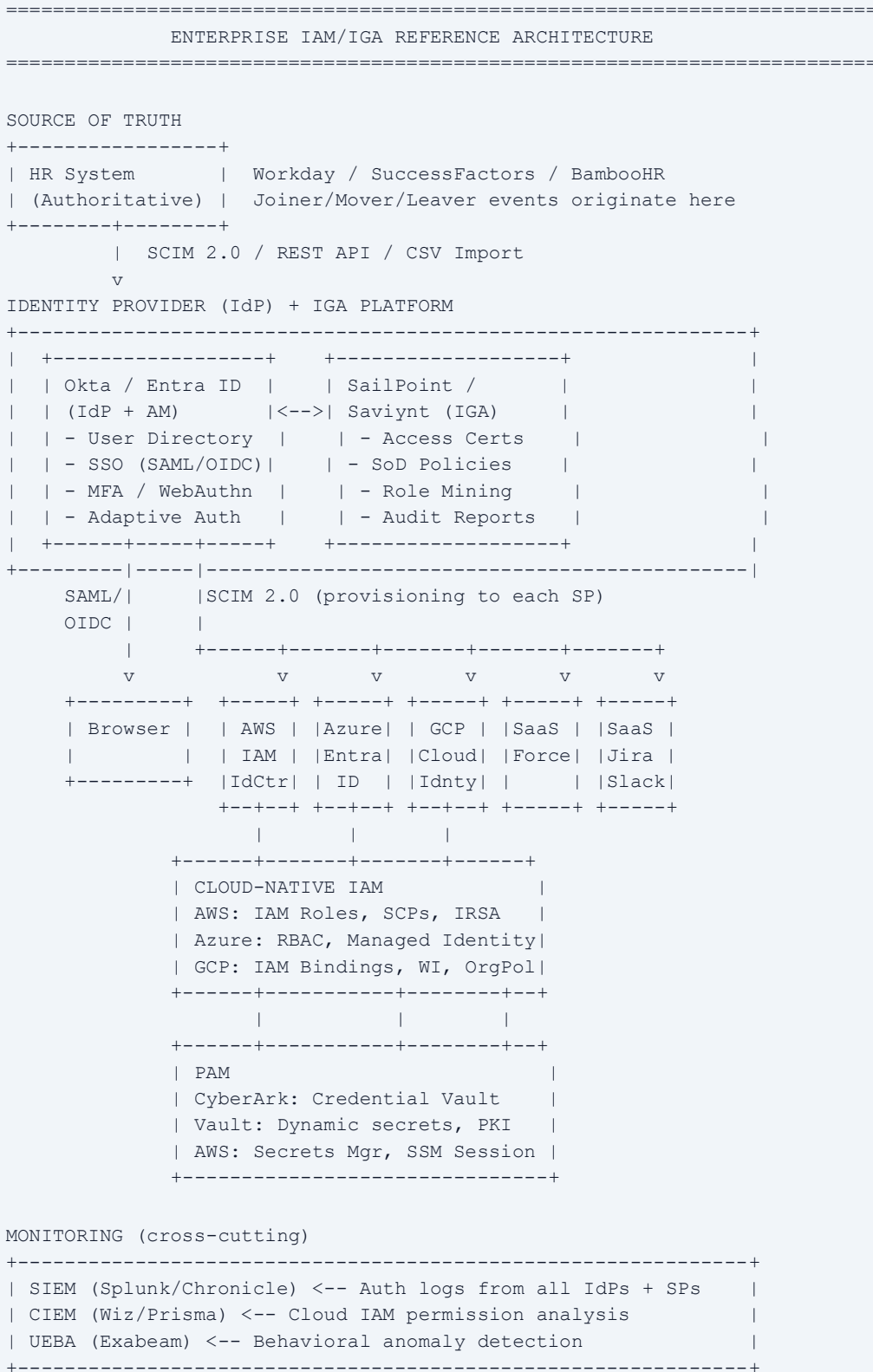
13. Enterprise IAM/IGA Reference Architecture

This section provides the end-to-end architecture that ties every previous concept together. At Staff/Principal level, you must be able to whiteboard this architecture and explain every data flow, integration point, and failure mode.

13.1 The Five Pillars of Enterprise IAM

Pillar	What It Answers	Key Capabilities	Primary Tools
Identity Lifecycle Mgmt	Who exists? When created/modified/disabled?	JML automation, HR-driven provisioning, account reconciliation, orphaned account detection	Okta Lifecycle Mgmt, SailPoint IdentityNow, Entra ID Governance, Saviynt
Access Management (AM)	How do users authenticate and authorize?	SSO (SAML/OIDC), MFA (WebAuthn), adaptive auth, session management, step-up auth	Okta, PingIdentity, Entra ID, ForgeRock, AWS IAM Identity Center
Identity Governance (IGA)	Who has access to what? Is it appropriate?	Access certification, SoD, role mining, policy enforcement, audit reporting	SailPoint, Saviynt, Okta IGA, One Identity
Privileged Access Mgmt (PAM)	Who has admin/root access? How monitored?	Credential vaulting, session recording, JIT access, privilege elevation, secrets rotation	CyberArk, BeyondTrust, Delinea, HashiCorp Vault
CIEM	What cloud permissions exist? Are they needed?	Overprivilege detection, least-privilege recs, multi-cloud permission analysis	Wiz, Prisma Cloud, CrowdStrike, AWS IAM Access Analyzer, GCP IAM Recommender

13.2 End-to-End Architecture (Whiteboard-Ready)



13.3 Data Flow Summary

Flow	Protocol	Data Carried	Frequency
HR → IdP	SCIM 2.0 or REST	User attributes: name, dept, title, manager, start/end date	Real-time or every 15 min
IdP → SPs (provisioning)	SCIM 2.0	Create/update/disable accounts + group memberships	Event-driven + nightly reconciliation
IdP → SP (authentication)	SAML 2.0 or OIDC	Identity assertion: NameID, groups, auth context	On every user login
K8s → Cloud IAM	OIDC token exchange	K8s SA OIDC token → cloud temp credentials	On every API call (cached 1h)
IGA → IdP	SCIM / API	Access certification decisions: approve/revoke	During quarterly campaigns
PAM → Targets	SSH/RDP/API	JIT credentials, session recordings	On privileged access requests
All → SIEM	Syslog/API/CloudTrail	Auth events, authz decisions, provisioning actions	Continuous streaming

14. IGA & PAM Tool Landscape

Interviewers expect you to know the tools, not just the protocols. This section covers major vendors, strengths, and when to choose each.

14.1 IGA Platforms

Platform	Deployment	Core Strengths	Best For
SailPoint IdentityNow / IdentityIQ	Cloud / On-prem	Most mature IGA. Deep access certification. AI role mining. 200+ connectors. Strong SOX/HIPAA reporting.	Large enterprises (5K+ employees) with complex compliance requirements.
Saviynt Enterprise Identity Cloud	Cloud-native	Strong CIEM integration. Converged IGA + PAM + App GRC. Good multi-cloud visibility. SoD policy engine.	Cloud-first enterprises needing IGA + CIEM in one platform. Financial services.
Okta Identity Governance	Cloud-native	Natively integrated with Okta IdP. Simple access request/approval workflows. Fast to deploy (weeks).	Companies already on Okta needing basic-to-moderate governance.
Microsoft Entra ID Governance	Cloud (Azure)	Deep M365/Azure integration. Entitlement Management with access packages. Lifecycle Workflows.	Microsoft-centric environments. Enterprises on Entra ID Premium P2.
One Identity Manager	On-prem / Cloud	Strong AD and SAP integration. Covers IGA + PAM + AD management.	SAP-heavy environments. Large AD forests.

14.2 PAM Platforms

Platform	Core Capabilities	Best For
CyberArk Privilege Cloud	Credential vaulting (auto-rotation), session isolation/recording, JIT access, threat analytics, Conjur for DevOps	Large enterprises with regulatory requirements (PCI, SOX). Thousands of privileged accounts.
HashiCorp Vault	Dynamic secrets (DB creds on-demand), PKI, encryption-as-a-service, K8s-native auth, transit engine	Cloud-native orgs with DevOps culture. Short-lived secrets for microservices and CI/CD.
BeyondTrust	Endpoint privilege management (remove local admin), privileged remote access, password safe	Endpoint privilege reduction. Mixed Windows/Linux environments.
Delinea (Thycotic + Centrify)	Secret Server (vault), Privilege Manager (endpoint), Server Suite (AD bridge for Unix)	Mid-market needing straightforward PAM. Heavy Unix/Linux infrastructure.
AWS Secrets Manager + SSM Session Manager	Native AWS secrets rotation (RDS, Redshift). SSM replaces SSH/RDP bastion. IAM-controlled.	AWS-only organizations. Serverless and container environments.

14.3 Tool Selection Matrix

Company Profile	Recommended Stack	Why
Startup (50 employees), AWS-only, SOC 2	Okta + AWS IAM Identity Center + AWS Secrets Manager	Fastest to deploy. No need for SailPoint complexity.
Mid-market (500), multi-cloud, HIPAA	Okta + Okta IGA + HashiCorp Vault + Wiz	Okta IGA for access reviews. Vault for dynamic secrets. Wiz for CIEM.
Enterprise (5K+), hybrid, SOX + PCI	Okta + SailPoint + CyberArk + Wiz	SailPoint for complex certifications. CyberArk for session recording.
Financial services (10K+), heavy regulation	Okta/Ping + SailPoint IQ + CyberArk + Saviynt + Splunk	Maximum governance. Saviynt for SoD. Splunk UEBA for identity analytics.

15. Access Control Models: RBAC, ABAC, PBAC, ReBAC

"When would you choose ABAC over RBAC?" is one of the most common Staff/Principal interview questions. Here are all four models with cloud-specific implementations.

Model	How It Works	Cloud Implementation	Best For	Limitation
RBAC (Role-Based)	Users assigned to roles. Roles have permissions. User gets union of all roles' permissions.	AWS: IAM Roles + Permission Sets. Azure: RBAC built-in/custom roles. GCP: Predefined/custom IAM roles. K8s: ClusterRole/RoleBindings.	Stable job functions. <500 roles. Clear permission hierarchy.	Role explosion: 200 teams × 10 apps × 4 envs = 8,000 roles.
ABAC (Attribute-Based)	Access decisions based on attributes of user (dept, clearance), resource (classification), action, and environment (time, IP).	AWS: IAM conditions (PrincipalTag/ResourceTag). GCP: IAM Conditions (resource.matchTag). Azure: Conditional Access + ABAC.	Large orgs. Frequent team changes. Tag-driven access.	Complex policy evaluation. Hard to audit. Performance overhead.
PBAC (Policy-Based)	Explicit policy rules evaluated by a policy engine. Superset of RBAC+ABAC.	OPA/Rego on K8s admission controllers. AWS Cedar (Verified Permissions). Envoy ext-authz.	Microservices needing fine-grained context-aware authz. Multi-cloud consistency.	Policy engine in request path (latency). Rego learning curve.
ReBAC (Relationship-Based)	Access based on user-to-resource relationships (owner, member, viewer).	Google Zanzibar (internal). SpiceDB, OpenFGA (CNCf). AWS Verified Permissions (Cedar supports ReBAC).	Document/file sharing. Multi-tenant SaaS. Resource ownership hierarchies.	Requires relationship graph DB. Complex modeling.

15.1 AWS ABAC Example

```
// IAM Policy: Developer manages only resources tagged with their team
{
  "Effect": "Allow",
  "Action": ["ec2:StartInstances", "ec2:StopInstances", "s3:GetObject"],
  "Resource": "*",
  "Condition": {
    "StringEquals": {
      "aws:ResourceTag/Team": "${aws:PrincipalTag/Team}",
      "aws:ResourceTag/Env": "${aws:PrincipalTag/Environment}"
    }
  }
}
```

```
// Developer tagged Team=payments, Env=dev gets access to ALL resources  
// tagged Team=payments, Env=dev. No new roles needed for new teams.
```

16. Cloud-Specific IAM Architectures

Each cloud has its own IAM model, primitives, and patterns. This section covers end-to-end IAM for AWS, Azure, GCP, and multi-cloud.

16.1 AWS IAM Architecture (200-Account Organization)

```
AWS ORGANIZATION
+-- Management Acct (root, billing, SCPs only)
|   SCPs: deny public S3, deny IMDSv1, restrict regions
|
+-- Security OU
|   +-- Security Tooling (GuardDuty admin, Security Hub, CloudTrail org trail)
|   +-- Log Archive (immutable S3, Object Lock, read-only)
|
+-- Identity OU
|   +-- IAM Identity Center (federated with Okta via SAML)
|       SCIM provisions groups -> Permission Sets
|       Permission Sets: Admin, Developer, ReadOnly, SecurityAudit
|       ZERO IAM users anywhere
|
+-- Prod OU (SCPs: deny iam:CreateUser, deny iam:CreateAccessKey)
|   +-- Prod Accounts 1..N (IAM Roles only, IRSA for EKS)
|
+-- Non-Prod OU (relaxed SCPs, same role patterns)
    +-- Dev/Staging Accounts
```

AWS IAM Primitive	What It Is	When to Use
IAM Role	Temporary credentials assumed by any principal	Always. Every workload gets a role. No static keys.
SCP	OU-level deny guardrail. Cannot be overridden.	Org-wide baselines: deny public S3, deny IMDSv1, restrict regions.
Permissions Boundary	Max permissions ceiling for a role	Delegated admin: devs create roles capped at non-admin level.
IRSA / Pod Identity	K8s SA → IAM Role via OIDC token exchange	Every EKS pod. Replaces instance profiles.
IAM Access Analyzer	Finds overly broad or public policies	Always-on. Generate least-privilege from CloudTrail usage.

16.2 Azure / Entra ID Architecture

```
AZURE ARCHITECTURE
+-- Entra ID Tenant (federated with Okta or native)
|   +-- Conditional Access Policies:
|       |   Require MFA all users, compliant device for admins,
|       |   block legacy auth, restrict by location
|   +-- Privileged Identity Management (PIM):
|       |   JIT activation (Global Admin eligible, not permanent),
|       |   time-limited (4h max), approval workflow, access reviews
|   +-- Entitlement Management:
```

```
|      Access Packages, self-service request, auto-expire 90 days
|
+-- Management Groups (hierarchy for Azure RBAC + Policies)
    Root MG -> Platform MG -> Landing Zone MGs
    Azure Policies (equivalent of SCPs) at MG level
```

16.3 GCP IAM Architecture

```
GCP ARCHITECTURE
+-- Organization (acme.com)
|   Org Policies: disableServiceAccountKeyCreation,
|   requireOsLogin, allowedPolicyMemberDomains=[acme.com]
|
|   +-- Folders (Prod, Non-Prod, Security)
|   +-- Projects (IAM boundary)
|       IAM Bindings: user/SA -> role -> resource
|       Workload Identity for GKE (OIDC -> STS -> GCP SA token)
|       VPC Service Controls (data perimeter around projects)
```

KEY GCP DIFFERENCES FROM AWS:

- IAM bindings are additive (no explicit deny -- deny only via Org Policy)
- SA keys are #1 GCP attack vector (disable via Org Policy)
- IAM Recommender generates least-priv from 90-day audit logs

16.4 Multi-Cloud IAM Architecture

Layer	Multi-Cloud Solution	Implementation
Single IdP	Okta or Entra ID as ONE IdP for all clouds	Okta → SAML to AWS Identity Center. Okta → SAML to Azure Entra ID (B2B). Okta → OIDC to GCP Cloud Identity. One MFA policy. One deprovisioning path.
Machine Identity	OIDC federation + short-lived tokens everywhere	AWS: IRSA. GCP: Workload Identity. Azure: Managed Identity. Cross-cloud: GCP WIF accepting AWS OIDC tokens.
Policy-as-Code	OPA with shared Rego policies	Same Rego in EKS + GKE admission controllers. Cloud-native guardrails (SCPs, Org Policies, Azure Policies) for cloud-specific rules.
CIEM	Wiz or Prisma Cloud across all clouds	Single pane: overprivileged identities, unused permissions, toxic combinations across all three clouds.
Logging	All audit logs → single SIEM	CloudTrail → S3. GCP Audit Logs → Pub/Sub. Azure Activity Log → Event Hub. All → Splunk/Chronicle.
PAM	CyberArk or Vault as cross-cloud platform	Vault with auto-unseal per cloud. AWS/GCP/Azure secrets engines. One policy controls all cloud credentials.

Multi-cloud anti-pattern #1: Anti-pattern: Each team picks their own IdP per cloud. Result: three directories, three MFA policies, three deprovisioning paths. A terminated user keeps access in the cloud where someone forgot to disable. SINGLE IdP is non-negotiable.

17. The WH Question Framework for IAM/IGA

Staff/Principal interviewers structure probes around the WH framework. For every IAM topic, you should answer all five:

Topic	WHAT?	WHERE?	WHICH tool?	WHO manages?	HOW technically?
SSO	One login → multiple apps	All enterprise SaaS, cloud consoles	SAML/OIDC via Okta, Entra ID, Ping	IAM team configures. Security sets MFA policy.	IdP issues signed assertion. SP validates signature + conditions. Session created.
MFA	Second factor beyond password. Phishing-resistant = domain-bound.	Every login. Required for admin. Step-up for sensitive ops.	WebAuthn (phishing-proof), TOTP (legacy), Push (MFA fatigue risk)	Security team mandates. Users self-enroll. Help desk handles lockouts.	WebAuthn: browser sends challenge to authenticator. Signed with private key + origin. Server verifies.
Provisioning	Automated user account CRUD from HR source of truth	Every SaaS app, cloud platform, internal tool	SCIM 2.0 via Okta/SailPoint connectors	HR owns lifecycle. IAM configures pipelines. IGA monitors drift.	HR event → IdP → group rules → SCIM PATCH to each SP. Nightly reconciliation.
Access Certification	Periodic review: is each user's access still appropriate?	SOX, HIPAA, PCI regulated environments. Quarterly.	SailPoint, Saviynt, Okta IGA, Entra ID Reviews	IGA team configures. Managers review. Audit validates.	IGA pulls entitlements. Campaigns generated. Managers approve/revoke. Executed via SCIM.
PAM	Control + monitor admin/root/superuser access	DB admin, cloud consoles, prod SSH, CI/CD secrets	CyberArk, Vault, AWS Secrets Mgr, Azure PIM	Security owns platform. Platform team manages secrets. SOC monitors.	JIT: request → approval → PAM checks out cred → session recorded → cred rotated → expires.
SoD	Prevent conflicting permissions on one person	Financial apps (SOX). Cloud IAM (CreatePolicy + AttachPolicy).	SailPoint SoD, Saviynt, OPA policies	Compliance defines rules. IGA enforces. Violations reported.	Conflict matrix: Role A conflicts with Role B. Flagged during provisioning/certification.
CIEM	Analyze cloud permissions for overprivilege	All cloud environments, especially post-migration	Wiz, Prisma, AWS Access Analyzer, GCP Recommender	Cloud security reviews. Platform remediates. CISO reports.	Ingest CloudTrail 90 days. Compare granted vs used. Generate least-privilege recommendations.

Topic	WHAT?	WHERE?	WHICH tool?	WHO manages?	HOW technically?
Zero Trust	Never trust, always verify. Every request authenticated regardless of network.	Remote workforce, cloud-first, perimeter-less	OIDC + WebAuthn + OAuth + mTLS. Verified Access, Conditional Access.	Security arch designs. IAM implements. Every team adopts.	(1) Verify identity + MFA (2) Check device posture (3) Evaluate context (4) Grant scoped access (5) Log everything.

18. End-to-End Multi-Cloud IAM Scenarios

These are whiteboard-level scenarios for Staff/Principal architecture design rounds.

Scenario 1: Greenfield Multi-Cloud IAM (2,000 employees, HIPAA)

Context: Healthcare company deploying on AWS (primary) and GCP (analytics). 2,000 employees, 300 contractors, 50 SaaS apps. Current state: on-prem AD, no SSO, manual provisioning, shared admin passwords on a wiki.

- 1. IdP:** Okta as cloud IdP. AD Agent syncs from on-prem. SAML SSO for all 50 SaaS apps (90-day rollout, prioritize by user count). OIDC for new custom apps.
- 2. MFA:** WebAuthn via YubiKeys for all staff, passkeys for contractors. Policy: WebAuthn required for prod cloud + admin roles. Push MFA for low-sensitivity apps (deprecate in 12 months).
- 3. AWS:** Okta → SAML → IAM Identity Center. SCIM provisions Permission Sets. SCP at Prod OU: deny iam:CreateUser, deny public S3, deny IMDSv1, restrict regions.
- 4. GCP:** Okta → SAML → Cloud Identity. Org Policies: disableServiceAccountKeyCreation, allowedPolicyMemberDomains. Workload Identity for GKE. VPC Service Controls around BigQuery with PHI.
- 5. IGA:** Okta Identity Governance for HIPAA access reviews. Quarterly certification of AWS/GCP admin roles. Dual-approval (manager + data owner) for PHI access.
- 6. PAM:** HashiCorp Vault for dynamic DB creds, SSH cert signing, CI/CD secrets. CyberArk for session recording of prod admin access (HIPAA audit requirement).
- 7. Monitoring:** CloudTrail + GCP Audit Logs → Splunk. Wiz for CIEM. Alert on: IAM user created (never allowed), SA key created (never allowed), role assumption from unexpected IP.

Scenario 2: Post-Breach IAM Remediation

Context: Compromised developer laptop → exfiltrated Okta session cookies → attacker accessed AWS Console via SSO → created IAM user with AdministratorAccess. Detected 4 hours later by GuardDuty.

Immediate (30 min): (1) Clear all Okta sessions for the user. (2) Delete attacker-created IAM user. (3) Attach deny-all SCP to compromised account. (4) Rotate user password + deregister MFA devices. (5) CloudTrail analysis: map all attacker API calls and blast radius.

Architectural fixes: (1) SCP: deny iam:CreateUser and iam:CreateAccessKey org-wide. (2) Okta session binding to device fingerprint. (3) EventBridge rule: any iam:CreateUser → immediate PagerDuty alert. (4) Deploy device trust: Okta FastPass or AWS Verified Access with MDM posture checks.

Scenario 3: Contractor Lifecycle in Multi-Cloud

Context: 50 contractors/month from 10 agencies. Need access to specific AWS accounts, Azure DevOps, and Jira. Must auto-lose access when contract ends.

- 1. Federated Identity:** Each agency's IdP federated with Okta (SAML). Agency authenticates their contractors. Okta applies attribute-based group mapping: vendorName + contractEndDate > today → group Contractors-AgencyX.
- 2. Time-Limited:** contractEndDate synced from procurement system. Okta auto-deactivates on expiry. No manual intervention.
- 3. Scoped Permissions:** Group maps to: AWS ReadOnly Permission Set (specific accounts), Azure DevOps Stakeholder role, Jira Viewer. All via SCIM.
- 4. Audit:** Quarterly access review in Okta IGA. Agency contract managers certify active contractors. Uncertified = auto-deactivated. SCIM propagates in 60 seconds.

All content in this document is based on published RFCs, OASIS standards, W3C specifications, and documented cloud provider documentation. Protocol capabilities and cloud service features evolve — verify against current documentation before citing in interviews.